

Article

Making a Real-Time IoT Network Intrusion-Detection System (INIDS) Using a Realistic BoT-IoT Dataset with Multiple Machine-Learning Classifiers

Jawad Ashraf ¹, Ghulam Musa Raza ², Byung-Seo Kim ^{2,*}, Abdul Wahid ³ and Hye-Young Kim ⁴

¹ School of Electrical Engineering and Computer Science, National University of Sciences and Technology, Islamabad 44000, Pakistan; mashraf.dphd19seecs@seecs.edu.pk

² Department of Software and Communications Engineering, Hongik University, Sejong 30016, Republic of Korea; ghulammusaraza96@gmail.com

³ School of Computer Science, University of Birmingham, Birmingham B15 2TT, UK; a.wahid@bham.ac.uk

⁴ Department of Games, School of Games, Hongik University, Sejong 30016, Republic of Korea; hykim@hongik.ac.kr

* Correspondence: jsnbs@hongik.ac.kr; Tel.: +82-41-860-2539

Abstract: Cyber-attacks have become a significant concern today, particularly in IoT environments where security poses a substantial challenge due to the distributed nature and heterogeneity of protocols. To efficiently detect threats in IoT networks, it is crucial to develop a robust intrusion-detection system (IDS) capable of identifying various modern and traditional attacks with high accuracy. Most existing machine-learning-based intrusion-detection systems for IoT have been trained using outdated datasets that do not accurately reflect IoT scenarios. Additionally, current research does not adequately address which machine-learning classifiers are most suitable for developing an efficient IDS in IoT environments. In our research, we have developed and trained a real-time intrusion-detection system for IoT networks that can detect multiple modern and traditional threats with high accuracy. We created seven instances of real-time IDS using state-of-the-art machine-learning algorithms, including Logistic Regression, Support Vector Machine, K-Nearest Neighbors, Decision Tree, Random Forest, Naïve Bayes, and Artificial Neural Networks. Using the Pearson Correlation Coefficient, we extracted the most relevant features from the BoT-IoT dataset. After rigorous preprocessing, we used these data to train our algorithms. Our trained model, INIDS, is not only up to date and real-time but also capable of accurately identifying multiple categories of attacks specifically related to IoT networks. To achieve maximum accuracy, instead of selecting only one classifier, we evaluated seven advanced machine-learning algorithms and provided a comprehensive comparison of their performance and efficiency in the context of IoT networks. This analysis can guide future researchers in choosing the right machine-learning algorithms for developing IDS. We found that Random Forest is the most robust classifier for IoT-based network intrusion-detection systems, achieving an accuracy of 99.2%. The second-best performer is Naïve Bayes, with an accuracy of 98.8%.

Keywords: intrusion detection; anomaly detection; IoT; machine learning; cyber-attacks; network security; BoT-IoT



Academic Editor: David Megías

Received: 25 December 2024

Revised: 27 January 2025

Accepted: 9 February 2025

Published: 15 February 2025

Citation: Ashraf, J.; Raza, G.M.; Kim, B.-S.; Wahid, A.; Kim, H.-Y. Making a Real-Time IoT Network Intrusion-Detection System (INIDS) Using a Realistic BoT-IoT Dataset with Multiple Machine-Learning Classifiers. *Appl. Sci.* **2025**, *15*, 2043. <https://doi.org/10.3390/app15042043>

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the Creative Commons

Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Intrusion is an activity of meddling in an information system to perform an action that is not legally allowed. An intrusion-detection system (IDS) is a system for identifying

intrusions and notifying them correctly to the proper authorities. With the proliferation of network devices, the size of network traffic is growing, which has led to an increase in the number of network attacks. Thus, the current network traffic situation has highlighted the importance of intrusion-detection systems [1,2].

As the Internet of Things (IoT) allows numerous and a variety of devices to connect, interact, and exchange data, security remains a big concern in IoT environments. Because of its distributed nature, IoT is prone to attacks and vulnerable to intruders. Due to the pervasiveness of IoT devices and the heterogeneity of protocols, it is easy for intruders to break into data and frustrate its integrity [3].

Presently, intrusion-detection systems (IDSs) have become indispensable to keep networks safe and secure. Such intrusion-detection systems (IDS) are meant to protect the network by spotting anomalous behaviors or illicit uses. Network intrusion-detection systems notice deliberate and successful attacks at the endpoints or within the networks. Currently, machine-learning algorithms have proved a promising approach to detecting intrusion. By applying supervised and unsupervised machine-learning algorithms to real-time datasets, we can detect the pattern of attacks, the nature of attackers, and the type and source of malicious data packets [4].

To date, many researchers have proposed different network intrusion-detection systems. Some of these proposed systems successfully detect network intrusions, but only a few of them have based their research on IoT scenarios, and only a few of them have performed their research experiments on real-time IoT datasets. The datasets used in current research have not actually originated from actual IoT networks.

For attack analysis and developing intrusion-detection systems, a number of datasets have been produced that are applicable in different research scenarios [5]. However, most of the existing intrusion-detection systems (IDS) have been designed using commonly available DARPA 98, KDD Cup 99, UNSW15, and NSL-KDD datasets. A comparison of these datasets has been given in Table 1. The problem is that none of these datasets genuinely resembles an IoT network. Moreover, these datasets were designed years ago. These outdated datasets no longer reflect the current network behaviors and have not undergone modern cyber-attacks.

Table 1. Comparison of datasets and advantage of Bot-IoT.

Dataset	Year of Creation	Realistic Traffic	IoT Traces	Full Packet Captured	Diverse Attack Scenarios	Labeled Data
DARPA98	1998	No	No	Yes	Yes	No
KDDCUP99	1999	No	No	Yes	Yes	Yes
CAIDA	2007	Yes	No	No	No	No
UNIBS	2009	Yes	No	Yes	No	No
ISCX	2012	Yes	No	Yes	Yes	Yes
UNSWNB15	2015	Yes	No	Yes	Yes	Yes
CICIDS-17	2017	Yes	No	Yes	Yes	Yes
BoT-IoT	2018	Yes	Yes	Yes	Yes	Yes

In our research, we have used a realistic BoT-IoT dataset [6]. This real-time dataset contains the most modern data attacks and offers us the ease to use it for the purpose of intrusion-detection system design and analysis using machine-learning techniques. With a huge training dataset, the machine-learning predictors have been trained thoroughly. Also, we have selected more and better features of malicious datasets. Moreover, along with other supervised machine-learning algorithms, we have also tried the K-means nearest

neighbor algorithm (KNN), which is a very effective machine-learning algorithm but rarely used by researchers to develop NIDS, probably due to its clustering problem [7].

The main focus of our efforts remains to accumulate a set of maximum relevant modern and traditional attacks from a real-time IoT dataset in order to bring the utmost accuracy to our model. For this purpose, we have used about three million records, which is a huge amount of data to train our model accurately. Of the used data, 80% have been used for training purposes, and the rest of the data have been reserved for testing purposes. The main purpose of using multiple ML classifiers is to figure out which classifier suits best in developing IoT-based NIDSs, so we have used Logistic Regression, Support Vector Machine, K-Nearest Neighbor, Decision Tree, Random Forests, and Naïve Bayes and drawn a performance comparison among them. Additionally, by using three sets of features, we have trained each of the models and observed the trends of overfitting and underfitting. The results of this research give a clear direction to researchers as to which type of trained model of IDS can be incorporated as a module in traditional IoT environments like smart cities, agriculture, ad hoc networks, and SDN-based networks.

Since the release of BoT-IoT, only a couple of noticeable works can be seen. An important work was completed by [8], who used Naïve Bayes, BayesNet, Decision Tree C4.5, and Random Forest. A bijective soft set method selects the most appropriate machine-learning classifier from a given set. However, the authors used a new BoT-IoT dataset with 44 features. In another work, the same authors admitted that the use of 44 features was too much and affected IDS performance [9]. Moreover, further research is needed to determine how much the bijective approach is effective in selecting an appropriate machine-learning algorithm for NIDS. The work in [10] used blockchain and smart contracts in IoT and cloud environments to detect intrusions. To check the performance of their model, the BoT-IoT dataset, along with UNSW-15, was used. Although their proposed method obviated the need for intermediaries for data security, it was still not effective in providing data privacy, as all transactions were publicly accessible. Another work of [11] used multiple supervised machine-learning algorithms and evaluated their performance. Their feature-selection method is not known. They stated that, for binary classification, the Random Forest algorithm performed the best.

In most of the related works, researchers employed only two or three popular classifiers. However, in our work, we have used seven supervised machine-learning algorithms, which, to the best of our knowledge, is the largest number of algorithms in any work pertaining to IoT-based intrusion detection. We used LR, SVM, KNN, DT, RF, NB, and ANN. Moreover, to analyze the behavior of different classifiers and their overfitting and underfitting trends, we used multiple feature sets and finally discovered the optimal number and types of features. Additionally, we used more data records for the better training of models. Actually, we used the whole 5% of data isolated by generators [6], which is a huge amount of data, comprising 3.6 million records, and in none of the previous works have that much data been used.

2. Materials and Methods

Our methodology consists of two phases. The first phase is the preprocessing of the huge BoT-IoT dataset, which involves the cleaning, filtration, and normalization of raw data. Appropriate feature selection is also integrated into the first phase. In the second phase, we use well-known supervised machine-learning algorithms for intrusion detection in real-world IoT data. We use Logistic Regression, Support Vector Machine, Decision Tree, Naïve Bayes, KNN, and Random Forest in our experiments. Finally, a comparison of the performances of all these classifiers is made. Figure 1 gives an overview of the steps involved in our work. The first step is the selection of records from the BoT-IoT

dataset. The raw data are available in the form of .csv files, which carries with it problems like imbalance in the dataset, data type mismatch, missing or null values, etc. These problems are resolved in the preprocessing steps. After cleaning of data, in the next step, we extract features and make three groups. Next, we feed the data into classifiers and train different models based on selected feature sets to predict malicious traffic in the network. The following sections describe all the experimental steps in detail.

2.1. Dataset Description

For our research, we used a new realistic BoT-IoT dataset designed by [6] at the University of New South Wales Canberra, Australia. It was designed specifically using different IoT devices in a smart-home configuration. These devices include a weather station that transmits temperature, humidity, and air pressure values, a smart refrigerator, a smart motion-based light, a smart door, and a smart thermostat. The gathered dataset is available in .csv files containing more than 72 million records and 46 extracted features. The dataset captures complete network information and includes both the normal IoT-related and other network traffic. Moreover, it contains different types of attack traffic normally used by botnets. Another important advantage of using these data is that they have already been labeled, giving us the label features that indicate an attack flow, the category, and the subcategory of attacks, which makes them easy to use for the purpose of classification. As it is a realistic dataset, it is highly imbalanced, and a high degree of correlation is present among features. Three types of attacks have been introduced in the system, namely, Information Gathering, Denial of Service, and Information Theft Attack. Each of the attacks has its own subcategories. Table 1 describes some important features of the BoT-IoT dataset and compares them with other publicly available datasets.

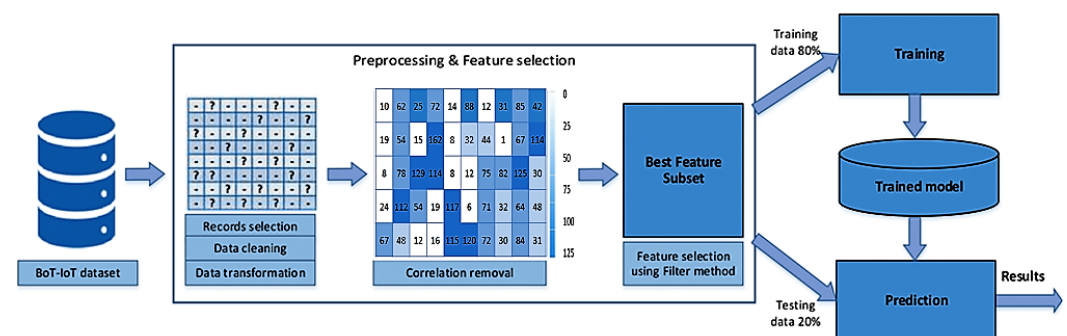


Figure 1. A representation of used methodology.

2.2. Record Selection and Dataset Balancing

The generated dataset is extremely large, containing more than 72 million records. As it is a real-time dataset, i.e., it has been collected over a realistic IoT network, it is a highly imbalanced dataset that mostly contains attack categories. Hence, we have selected about three million records, including an equal number of attack and normal categories. This huge amount of data has been used to train our model accurately.

Handling Class Imbalance

The BoT-IoT dataset, while highly comprehensive, presents a significant challenge in the form of severe class imbalance. This imbalance arises due to the overwhelming representation of certain attack categories and subcategories compared to others, which can lead to biased model training and poor generalization for underrepresented classes. To address this, we adopted a hybrid sampling approach to achieve class balance. Our hybrid sampling strategy combines both oversampling and undersampling techniques to effectively mitigate the imbalance without overfitting or discarding valuable data. Random oversampling

was applied to DDoS_HTTP and DoS_HTTP to amplify their representation in the dataset. Additionally, we utilized the Synthetic Minority Oversampling Technique (SMOTE) to generate synthetic samples of Keylogging and Data Theft classes, whose representation in data was very low, ensuring that the new data points are realistic and maintain the underlying data distribution. Simultaneously, random undersampling was employed for the majority classes to reduce their dominance while retaining the most representative samples. We carefully selected representative samples from the majority classes using random undersampling to preserve the diversity of the dataset and minimize information loss. By combining these two methods, our hybrid sampling approach achieves a balanced distribution across all classes, as shown in Figure 2.

Classes in the dataset		Instances in original dataset	Instances after class balancing	Technique applied
Information gathering	Service scanning	1,463,364	250,000	Random Undersampling
	OS Fingerprinting	358,275	250,000	Random Undersampling
Denial of Service	DDoS	TCP	19,547,603	Random Undersampling
		UDP	18,965,106	Random Undersampling
		HTTP	19,771	Random Oversampling
	DoS	TCP	12,315,997	Random Undersampling
		UDP	20,659,491	Random Undersampling
		HTTP	29,706	Random Oversampling
Information theft	Keylogging	1,469	250,000	SMOTE
	Data theft	118	250,000	SMOTE
Normal	Normal	9,543	500,000	SMOTE
Total		73,370,443	3,000,000	Hybrid Sampling

Figure 2. Records in the original BoT–IoT and Balanced datasets.

2.3. Data Preprocessing

Since the dataset used in our research contains realistic data, it inevitably contains inconsistency, redundancy, and missing values. Before using a dataset in a machine-learning model, it is imperative to convert it into a form that is cleaner and fit for analysis [12]. In our research, preprocessing involves data cleaning, data transformation, and feature selection by correlation removal.

2.3.1. Data Cleaning

The presence of redundant and anomalous records in the dataset can affect the learning of a model and give unrealistic accuracy due to overfitting [13], so such instances need to be removed in the preprocessing phase. For example, we have dropped out ‘saddr’ and ‘daddr’ features, which represent source and destination IP addresses. In our work, these features have been excluded because they mostly represent private IP addresses of packets, and packets usually have different IP addresses in such realistic datasets, so they have no value in training. Moreover, these two columns contain erroneous values. The ‘sport’ and ‘dport’ columns have been dropped because they contain the same values and have no value in class distinction. Two more features—‘stime’ and ‘ltime’—have not been included as they represent start packet time and end packet time, even though these represent important information. However, the reason for their elimination is that the same information has been provided by ‘Srate’ and ‘Drate’. Another feature, ‘daddr’, contains erroneous values, such as memory errors, so it was eliminated. We noticed that the dataset has many missing values. Missing values in data can make analysis difficult and can lead to decreased accuracy [14]. We have replaced all the missing values with zeros.

2.3.2. Data Transformation

Data transformation is an important part of preprocessing and is necessary for better training and decision-making. In the original dataset, ‘stddev’, ‘max’, and ‘mean’ are the features with continuous values, so we have converted them into integer type. Additionally,

the categorical values of ‘attack’ and ‘normal’ have been represented as 1 and 0, respectively. Moreover, the flattening of hierarchical data has been done in the same transformation phase. For example, an important feature, ‘category’, has subcategories like DoS, DDoS, normal, theft, and reconnaissance. We have flattened these hierarchies.

2.3.3. Feature Extraction and Grouping

Originally, the creators of the BoT-IoT dataset extracted 46 features, out of which they identified the 19 most significant features. Out of these 19 features, four were dropped during data cleaning. The remaining 15 features undergo preprocessing. The features ‘category’, ‘subcategory’, and ‘proto’ have sub-features. After flattening the hierarchy, we obtain 26 features as shown in Figure 3.

As BoT-IoT is a natural dataset, correlation among features is present. This correlation among features indicates redundancy and can lead to overfitting [15]. It is imperative to reduce or completely remove the correlation. Initially, some redundant and correlated columns were dropped in earlier preprocessing phases. However, the correlation among selected features is still fairly high. Figure 4 is a correlogram that shows both positive and negative correlations among features. We used the Pearson Correlation Coefficient to measure the linear relationship among the features of the BoT-IoT dataset. It gives a value on a spectrum between -1 to 1 . A value greater than zero signifies a positive relationship, while a value less than zero denotes a negative relationship. A high positive or negative value implies a stronger correlation among the features [16,17].

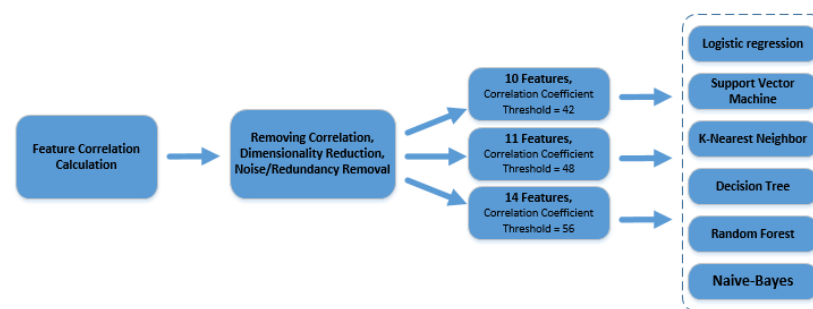


Figure 3. Feature selection using Pearson Correlation Coefficient.

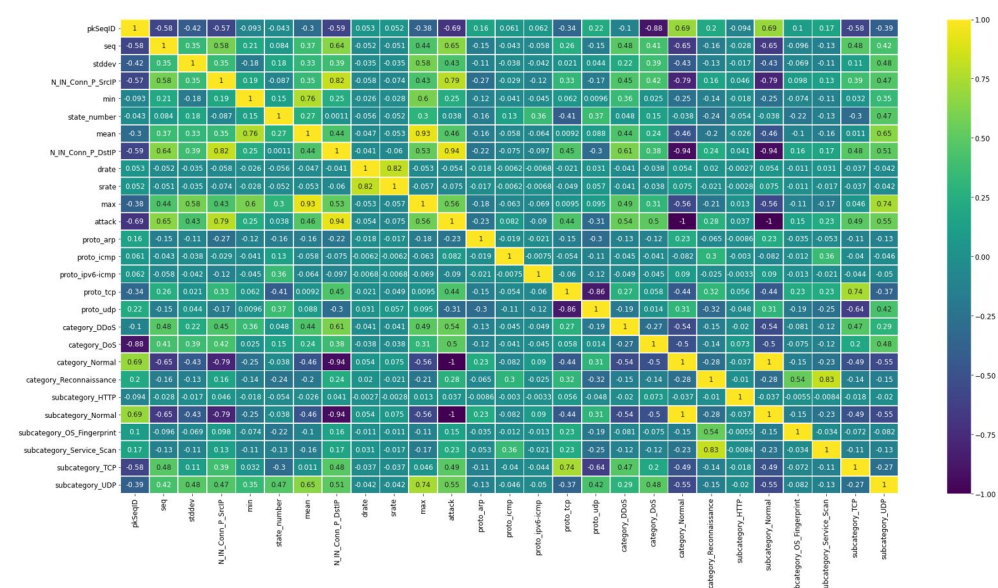


Figure 4. Correlation in original feature sets.

A feature is regarded as good if its correlation score with respect to other features is low [15,18]. This means that features are unrelated to each other and do not carry any redundant information.

We removed the features with a high correlation score. Thus, based on the correlation-removal method, we selected three sets of features. When the correlation threshold is set at 55, we obtain all the highly correlated features removed and obtain only 14 significant features out of 26. Table 2 gives the description of the final selected features after dropping insignificant and correlated columns. The correlogram in Figure 5 shows us the remaining features and their score. This set includes the following features: ‘pkSeqID’, ‘stddev’, ‘min’, ‘state_number’, ‘drate’, ‘proto_arp’, ‘proto_icmp’, ‘proto_ipv6-icmp’, ‘proto_tcp’, ‘category_DDoS’, ‘category_Reconnaissance’, ‘subcategory_HTTP’, ‘subcategory_OS_Fingerprint’, and ‘subcategory_UDP’.

Table 2. The best 14 features cited.

Feature	Description
pkSeqID	Row Identifier
Proto	Textual representation of transaction protocols present in network flow
Seq	Argus sequence number
Stddev	Standard deviation of aggregated records
N_IN_Conn_P_SrcIP	Number of inbound connections per source IP.
Min	Minimum duration of aggregated records
state_number	Numerical representation of feature state
Mean	Average duration of aggregated records
N_IN_Conn_P_DstIP	Number of inbound connections per destination IP
Drate	Destination-to-source packets per second
Srate	Source-to-destination packets per second
Max	Maximum duration of aggregated records
Category	Traffic category
Subcategory	Traffic subcategory

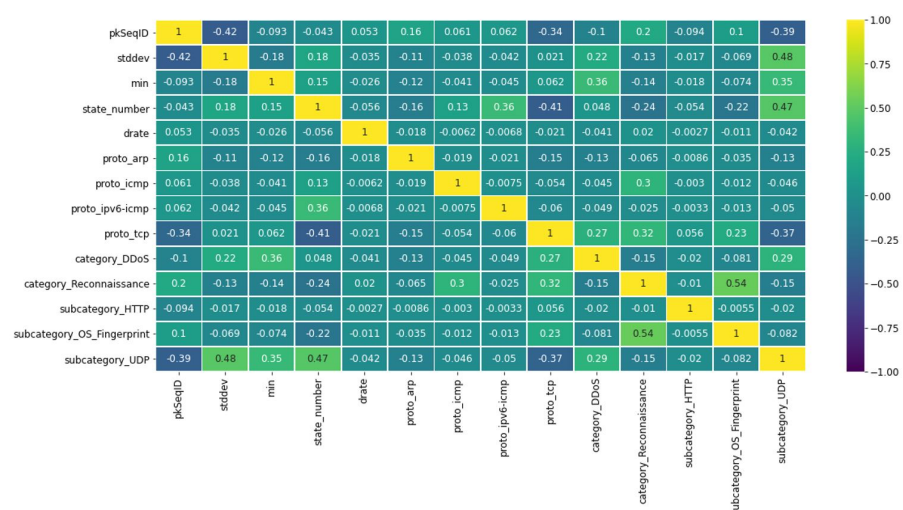


Figure 5. Correlation in 14-feature set.

By further decreasing the threshold to 45, we obtain 11 features. Figure 6 represents the correlogram of these features, and the features are as follows: ‘pkSeqID’, ‘stddev’,

'min', 'state_number', 'drate', 'proto_arpl', 'proto_icmpl', 'proto_ipv6-icmpl', 'proto_tcp', 'category_Reconnaissance', and 'subcategory_HTTP'.

Setting the correlation threshold value to 38 returns us a set of 10 features. The correlogram given in Figure 7 shows the obtained features. This set includes the following features: 'pkSeqID', 'min', 'state_number', 'drate', 'proto_arpl', 'proto_icmpl', 'proto_ipv6-icmpl', 'proto_tcp', 'category_Reconnaissance', and 'subcategory_HTTP'.

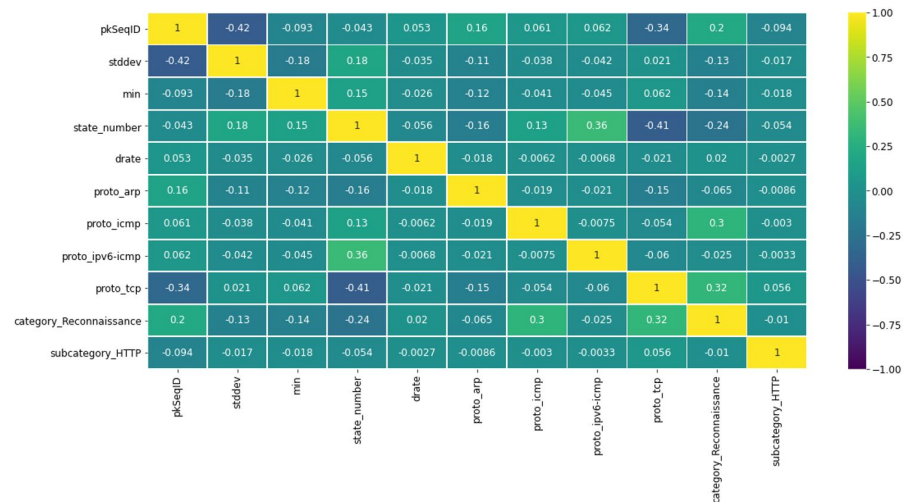


Figure 6. Correlation in 11-feature set.



Figure 7. Correlation in 10-feature set.

To further verify the selection of significant features, we conducted a feature-importance analysis using a Random Forest model. This approach not only provides a robust method for assessing feature relevance but also complements our correlation analysis. The resulting feature-importance plot, as shown in Figure 8, highlights the relative significance of each feature. Among the selected features, 'subcategory_UDP', 'stddev', 'subcategory_OS_Fingerprint', 'drate', 'state_number', 'proto_tcp', and 'category_DDoS' emerged as the most influential contributors to the model's predictions.

Other features, namely, 'proto_icmpl', 'proto_arpl', 'min', 'pkSeqID', 'category_Reconnaissance', 'subcategory_HTTP', and 'proto_ipv6-icmpl' also demonstrated considerable importance, reflecting their relevance in identifying specific attack subcategories and protocol-based variations. In contrast, 'mean', 'max', 'flags_number', 'sbytes', 'dbytes', 'spkts', 'sum', 'dpkts', 'dur', 'flags', 'daddr',

and ‘AR P Proto P Sport’ had nominal importance scores, suggesting no significant impact on the model’s performance, so all these features were eliminated.

This feature-importance analysis underscores the effectiveness of our feature-selection process and highlights the critical variables that contribute to the detection and classification of network events.

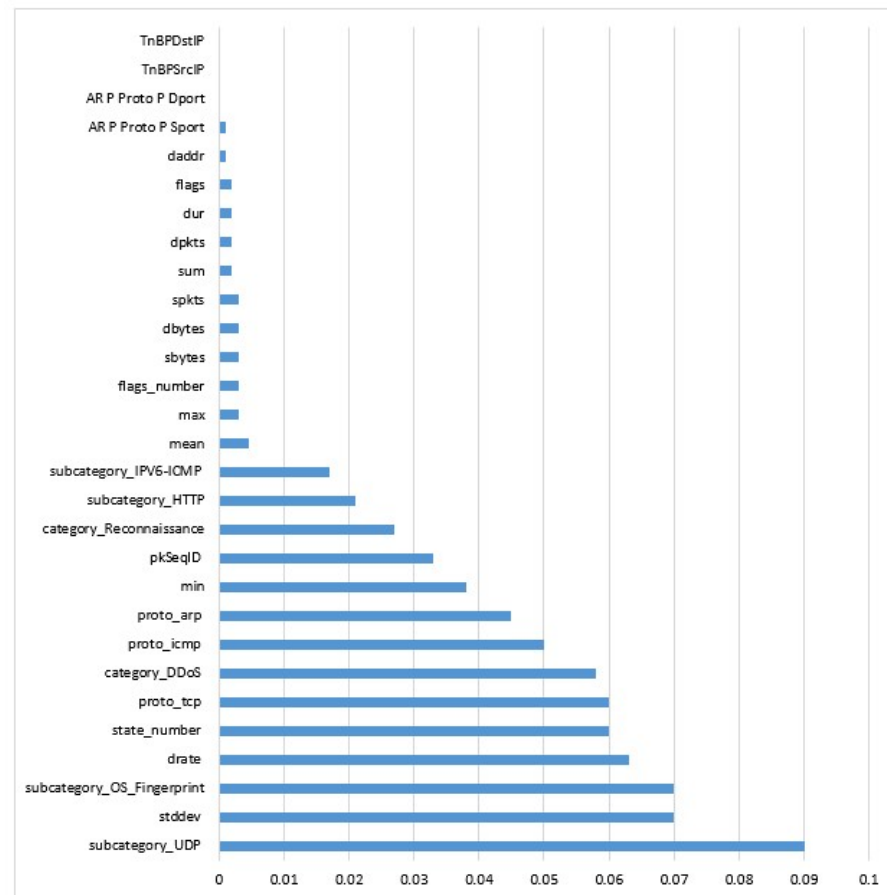


Figure 8. Feature-importance graph.

2.4. Machine-Learning Classifiers for IDS

In machine learning, several classification algorithms are available. In existing work, researchers have mostly used SVM, KNN, Decision Tree, and Artificial Neural Network on different publicly available datasets to solve network intrusion-detection problems [19–24]. However, due to the presence of irregularities in originally collected datasets and the high computational cost, it is quite difficult for researchers to determine a single optimal machine-learning algorithm that gives the best performance when used in IDS. To resolve this long-existing problem, we have tried six famous supervised machine-learning classifiers on the BoT–IoT dataset to discover which classifier is the most apt to develop IDS, particularly in the case of IoT networks.

Logistic Regression, being a fundamental classifying model, can discriminate data based on its quality. SVM is considered a split hyperplane that takes support vectors as input and maximizes the margin around the hyperplane or decision surface. The KNN algorithm takes labeled data as input to learning a function and gives an output when some unlabeled data are tried. The Decision Tree algorithm works based on building a tree. In DT, larger data are split into smaller parts until everything falls under the same category. By contrast, the RF algorithm uses a collection of decision trees that gives high execution speed. RF predicts by taking the average of the individual DT predictions. Naïve Bayes is a probabilistic model that allows the discovery of uncertainty about the model by finding

the probabilities of the results. It provides the ability to solve predictive and diagnostic problems. Artificial Neural Networks, inspired by the human brain, consist of connected nodes that build up a network. The connections between nodes have certain weights that need to be adjusted. ANN consists of an input layer, an output layer, and one or more hidden layers. In the results section, we compare these algorithms in terms of different parameters like Accuracy, Precision, Recall, and F1 Score [17,25].

2.5. Training and Testing

We trained all six above-mentioned classifiers using three feature sets, meaning that each model has been trained three times using a separate set of selected best features. The aim of using a different number of features is to check the trend of overfitting/underfitting and to verify which classifier is most robust and resistant to overfitting/underfitting. For training and testing, we split the data in a 4:1 ratio, where we used 80% of the data for training all the models, and 20% of the data were reserved for testing purposes. In this binary classification, the attacked data are denoted by 1, and benign data by 0.

3. Experimental Results and Analysis

As discussed earlier, we used three sets of features to deeply analyze the behavior of algorithms. Table 3 shows the comparison of accuracies of models for the original feature set along with extracted feature sets. We observe that 14 features is the optimal number of features that achieves the best accuracies for all classifiers used. The dropping of features has resulted in a significant decrease in accuracy. We can see in other sets of features that when the feature ‘subcategory_UDP’ has been dropped, then the accuracy also decreases significantly. Furthermore, by analyzing and comparing the accuracies, we can deduce the model that can give the best performance if used as an IoT network intrusion detector. From the achieved results, it is evident that the Random Forest algorithm outperforms all other algorithms. With an optimal feature set, it gives an accuracy of 99.2%. With other feature sets, it also achieved the highest accuracies. The second-best performing classifier is the Naïve Bayes algorithm, with an accuracy of 98.8%. Thus, by evaluating the accuracies of all the classifiers, we can obtain the idea of which classifier best suits NIDS concerning the IoT environment.

Table 3. Relative accuracies of different cited supervised learning algorithms.

Feature Set	LR	SVM	KNN	NB	DT	RF	ANN
Original feature set	0.877	0.899	0.910	0.932	0.888	0.948	0.885
14 features	0.926	0.963	0.971	0.988	0.961	0.992	0.931
11 features	0.852	0.879	0.946	0.956	0.826	0.968	0.914
10 features	0.831	0.850	0.928	0.948	0.803	0.958	0.899

Scalability and Computational Cost of the Proposed System

The proposed INIDS, trained using the Random Forest classifier, demonstrates exceptional accuracy (99.2%) and robustness, making it an ideal candidate for IoT-based intrusion detection. However, we recognize the importance of addressing its scalability and computational cost, especially in larger datasets and complex IoT environments. Random Forest is a powerful ensemble learning method that excels in handling high-dimensional data, as it combines multiple decision trees to deliver accurate predictions while reducing overfitting. Although computationally more expensive than simpler models, such as Logistic Regression or Naïve Bayes, its high accuracy and ability to handle non-linear

relationships justify its selection for real-time IDS in many practical scenarios. To assess scalability, we conducted experiments with progressively larger subsets of the BoT-IoT dataset. The results revealed that the training time and inference time of Random Forest increase linearly with dataset size. However, the computational requirements remain manageable, especially with modern computing resources, such as multicore processors or distributed computing systems. Moreover, techniques such as parallelization, dimensionality reduction, and optimized hyperparameter tuning can further enhance its efficiency and scalability.

In real-time IoT scenarios, where computational resources may be constrained, simpler models like Naïve Bayes, which achieved 98.8% accuracy in our experiments, can serve as a viable alternative when prioritizing speed over marginal gains in accuracy. However, in critical applications requiring the highest accuracy and reliability, the computational cost of Random Forest is a justified trade-off given its superior performance. This analysis highlights the versatility of INIDS in adapting to diverse IoT environments, with the choice of the classifier depending on the specific constraints and requirements of the application. Future work could further optimize Random Forest for resource-constrained devices by exploring techniques like pruning, incremental learning, or hybrid approaches that balance accuracy and computational efficiency. For detailed comparison in this regard, see Table 4.

Table 4. Training and testing time for machine-learning models on a dataset with 3 million samples and 14 features (32 GB RAM).

Model	Training Time (Seconds)	Testing Time (Seconds per 1M Samples)	Reasoning
Logistic Regression	20–60	2–5	Training involves solving a convex optimization problem; testing is fast (matrix multiplication).
Support Vector Machine	1000–3000+	100–300	SVM training scales poorly with large datasets (quadratic/cubic complexity); testing depends on support vector count.
K-Nearest Neighbors	~10 (Minimal training)	500–1000	Training is minimal (storing data); testing involves distance calculations for 3M training points.
Decision Tree	30–120	1–5	Training time depends on tree depth; testing is very fast as it involves traversing the tree.
Random Forest	300–1000+	10–50	Training scales with the number of trees and their depth; testing averages predictions across trees.
Naïve Bayes	5–15	1–3	Training and testing are very efficient due to simplicity (based on probabilities).
Artificial Neural Networks	2000–5000+	50–200	Training depends heavily on architecture, epochs, and batch size; testing requires forward passes for all samples.

4. Discussion

One very important observation during our implementation is that altering the number of features has a significant effect on accuracy. With fewer features, if the accuracy of an algorithm fluctuates less, then it is regarded as a robust algorithm. Such an algorithm can be trained with the minimum number of features giving high accuracy.

We can take the deviation of accuracy as a measure of the robustness of the algorithm. Figure 9 not only shows us overall accuracies but also shows us the aberrative behavior

of algorithms when encountered with the challenge of a smaller number of algorithms. From Table 3 and Figure 9, we can clearly see that the Random Forest not only gives the highest accuracy of all the algorithms but it is also worth noting that with all feature sets, it exhibits very little change in accuracy and all four accuracy columns are of nearly the same length. The same behavior can be observed for the Naïve Bayes algorithm, which also has higher accuracy and a little aberration in all the accuracies with respect to different feature sets. One important point can also be noted for ANN. Although ANN gives less accuracy as compared to RF and NB, it also shows little fluctuation in accuracy when encountering the challenge of a smaller number of available features. The reason is that ANN is an algorithm that can be trained with very small available information. By contrast, algorithms like Logistic Regression, KNN, and DT give us less overall accuracy and high aberration in accuracy with different feature sets.

Besides accuracy, we have further evaluated the performance of our models using parameters like Precision, Recall, F1 Score, and Specificity. Precision tells how precise a model is by determining the actual positive instances out of the predicted positive. The Recall parameter calculates the correct positive predictions out of all the desired positive predictions. The F1 Score is also an important metric that is used when unbalanced data are used. F1 Score weights Precision and Recall equally. Specificity gives the true positive rate. To correctly calculate the Recall and F1 Score, we have used a scale of 500 records. The reason for this scaling is that the dataset contains more attack records than normal traffic. To balance the normal (0) and attack (1) records, we have to scale our confusion matrices so that real Recall and F1 Score values can be obtained. Figures 9–15 and Table 5 show the comparison of results of all the performance metrics that have been calculated only for the optimal feature set. From these statistics, we see that, once again, RF gives the highest value of Recall, which is 0.993, and of F1 Score, which is 0.997. The second-best performance is exhibited by the Naïve Bayes, giving the Recall and F1 Score values as 0.989 and 0.994, respectively. Hence, from the obtained results and performance values, we can conclude that Random Forest and the Naïve Bayes algorithms of machine learning have a better classification performance when used in the scenario of real-time IoT networks. Because it is the first time a realistic IoT dataset has been generated and released publicly, further research on similar real-time IoT datasets must be done to confirm our claim in the future.

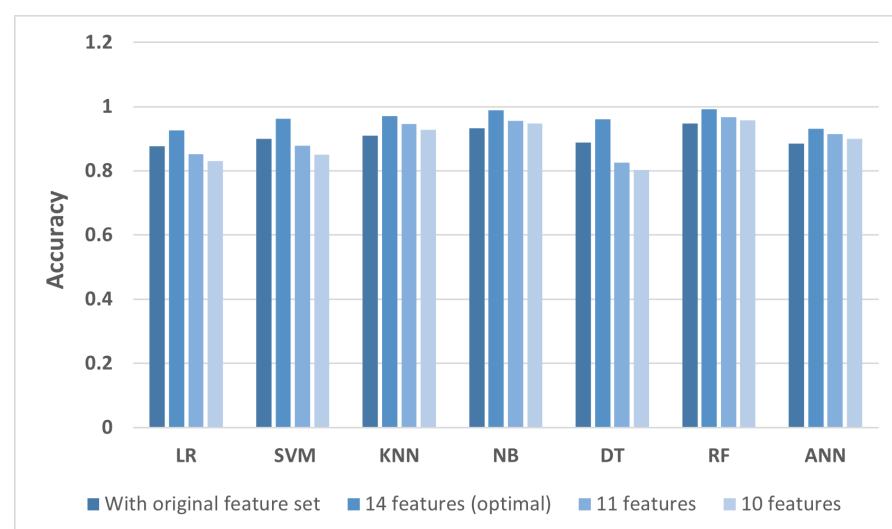


Figure 9. A comparison of achieved accuracies of all algorithms.

Machine-learning algorithms behave differently on the same type of data, and each machine-learning model works better on certain kinds of datasets than others. This is why

we have found which one works better with IoT data. We must now ask the question of how we compare different machine-learning algorithms. This depends on different aspects like forecast accuracy, sample complexity, bias-variance trade-offs, assumptions, and objectives, as well as the time and space complexity of the models. The predictive ability and performance of machine-learning models hence vary, and different performance parameters that rely on this fact also output different values.

It is worth discussing why the performances of each classifier are different from each other while using the same data and feature sets. This is because machine-learning algorithms are data-oriented, flexible, and adaptive. The output and different performance depend on many factors like the problem under consideration, the type and size of data, the number of features, the kind of output we want, speed and linearity, and, most importantly, their interpretability [26].

Table 5. Performance evaluation of classifiers.

Algorithm	Accuracy	Precision	Recall	F1-Score	Specificity	AUC
LR	0.926	1.00	0.931	0.964	1.00	0.77
SVM	0.963	1.00	0.964	0.981	1.00	0.78
KNN	0.971	1.00	0.972	0.986	1.00	0.85
NB	0.988	1.00	0.989	0.994	1.00	0.92
DT	0.961	0.955	0.971	0.963	1.00	0.79
RF	0.992	1.00	0.993	0.997	1.00	0.93
ANN	0.931	0.9763	0.935	0.955	1.00	0.80

4.1. Comparison with Other State-of-the-Art Works

So far, few researchers have produced considerable work regarding NIDS using the BoT-IoT dataset. Most relevant work has been put forth by [27], who chooses a single machine-learning model using his proposed bijective method. He uses a large number of features of data. This can lead to underfitting and large training time. In [11], the authors evaluated the performance of several supervised machine-learning algorithms in order to discover the most effective algorithm. Although they used the BoT-IoT dataset, their feature-selection method was inappropriate. They selected features by examining and observing data. Only a tried and tested feature-selection method can ensure the extraction of the most significant features. The work of [28] offers a novel NIDS pertaining to IoT networks using the BoT-IoT dataset. They used a deep-learning approach and generic features from field information in the packet. Their developed model can detect only DoS, DDoS, Reconnaissance, and Information Theft attacks. In [29], the authors used a Graph Neural Network (GNN)-based framework for IoT and trained their data using the BoT-IoT dataset. As we know, GNN has more time and space complexity, and hence, the model requires more time for training as well as space. Moreover, the work of [30] explores the capacity of various machine-learning algorithms in detecting the threats of IoT networks, but some of these models are not well trained and hence give low accuracy. The model presented by [31] Sarwar et al. (2022) is based on a modern IoT dataset, but they implemented only one classifier with fewer attack categories. Hence, their model cannot be regarded as robust. The Random Forest model employed by [18] is an SDN-based detector, but its accuracy is not remarkable, and it is trained using old datasets not representing present-day IoT scenarios. In a recent work [32], the authors trained a Deep Neural Network (DNN) to detect threats in an IoT environment, but its accuracy was low because the dataset used was not well balanced.

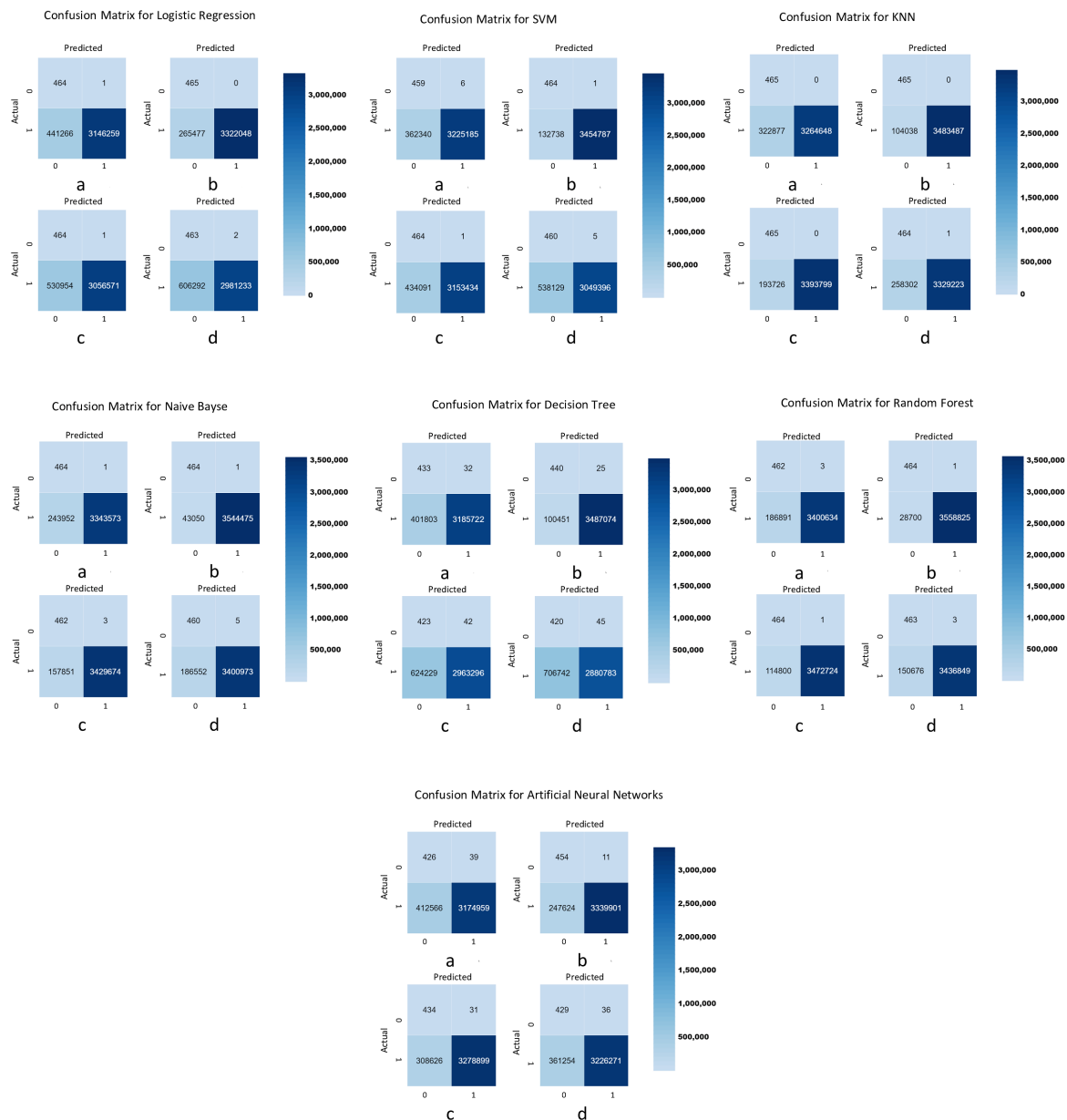


Figure 10. Confusion matrix of algorithms where, for each algorithm, (a) gives confusion matrix for full feature set (b) gives confusion matrix for 14 optimal features (c) gives confusion matrix for 11 features and (d) for 10 features.

4.2. Significance and Superiority of Our Work

The following points explicitly compare our work to prior studies, highlighting how our approach advances the state of the art:

4.2.1. Fully Utilizing the Potential of BoT-IoT

Most prior studies that used the BoT-IoT dataset did not fully utilize its potential. They often trained models using all features without proper feature selection or preprocessing, leading to suboptimal accuracy or resource-heavy models. Additionally, many studies only explored a limited subset of attack categories or focused on binary classification. By contrast, in our work, we systematically used the Pearson Correlation Coefficient to extract the most relevant features, reducing redundancy and improving model and resource efficiency. This allowed our IDS to focus on meaningful features, resulting in higher accuracy and faster

decision-making. Additionally, we used a huge dataset for training to bring about a robust and accurate IDS.

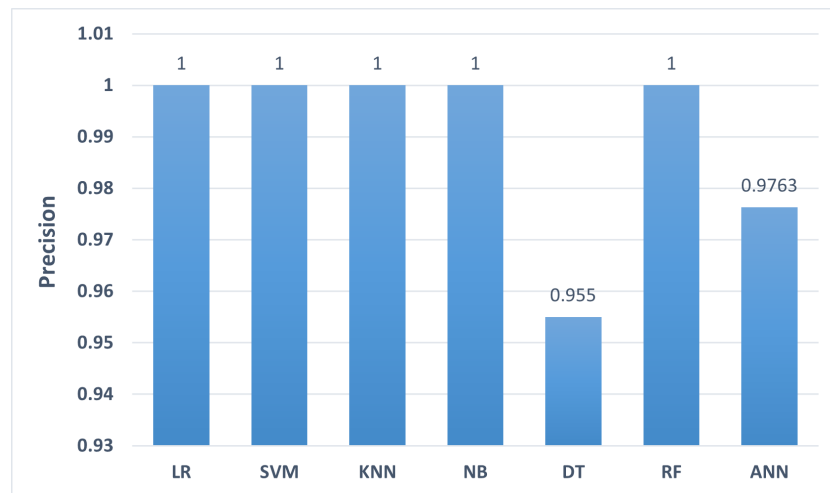


Figure 11. Precision results.

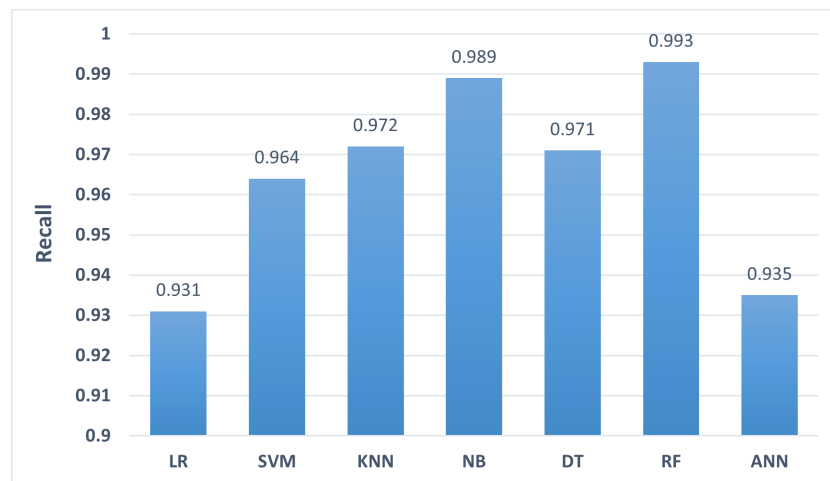


Figure 12. Recall results.

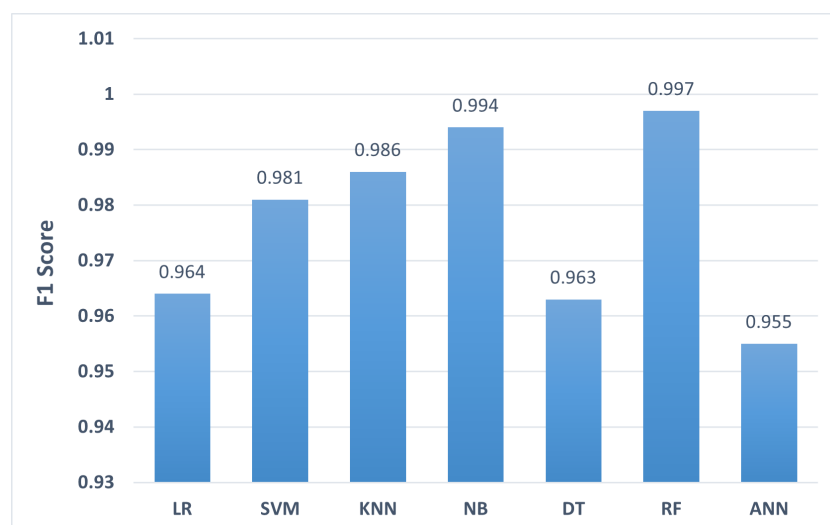


Figure 13. F1 Score results.

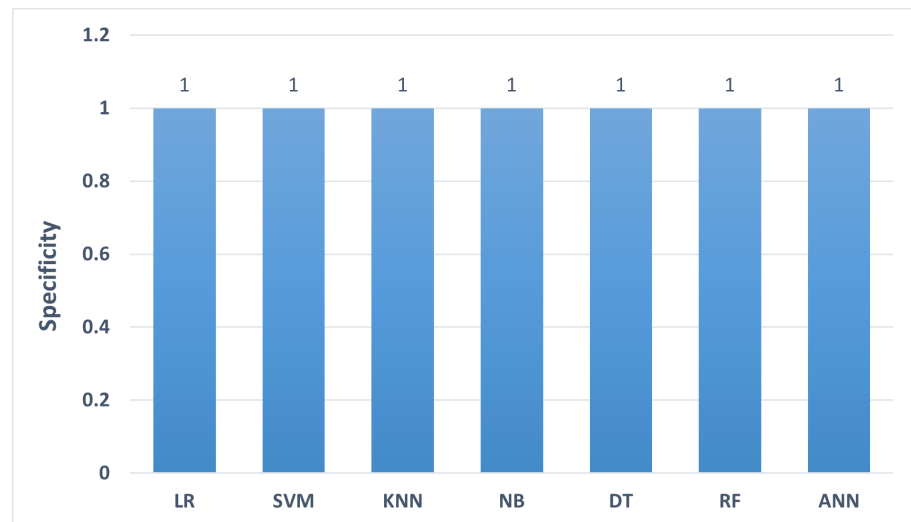


Figure 14. Specificity results.

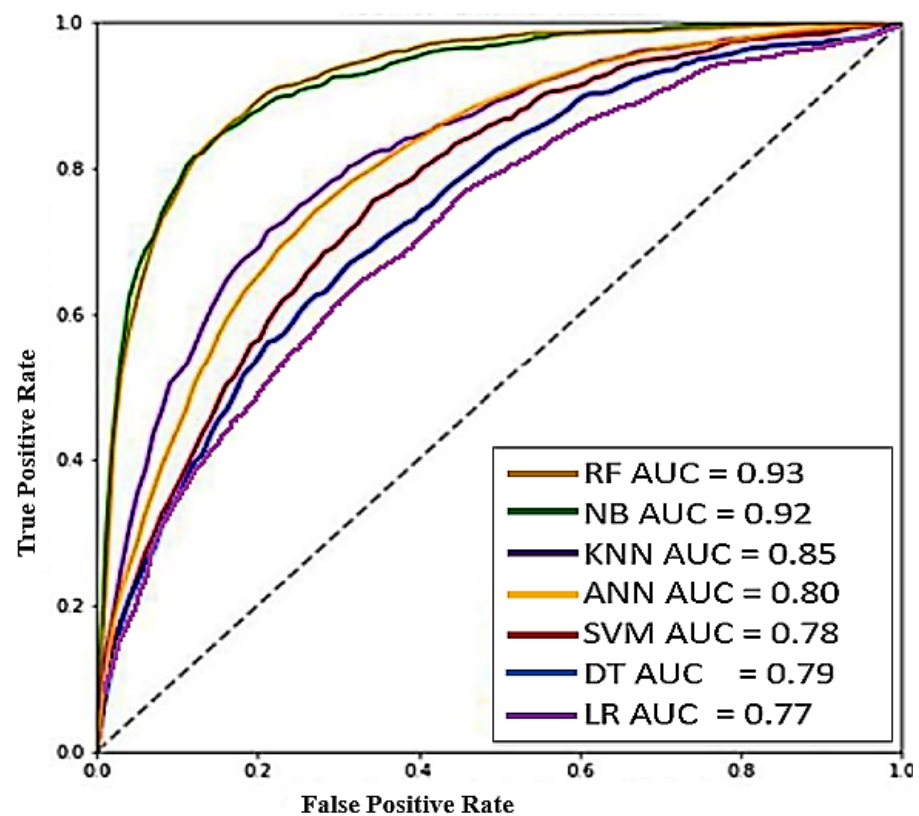


Figure 15. The AUC–ROC curve.

4.2.2. Systematic and Broad Classifier Evaluation

Many studies only test a single classifier or a small subset, leaving the performance of other machine-learning algorithms unexplored. Often, the selected classifiers are not rigorously compared in terms of accuracy, efficiency, or suitability for IoT-specific environments. We developed seven distinct IDS models using cutting-edge machine-learning algorithms (Logistic Regression, SVM, KNN, Decision Tree, Random Forest, Naïve Bayes, and ANN). We conducted an in-depth analysis of these algorithms, measuring their accuracy, suitability, and efficiency for IoT-specific intrusion detection. By identifying Random Forest as the most robust algorithm (99.2% accuracy) and Naïve Bayes as the second-best (98.8%), our research provides actionable guidance for future IDS development.

4.2.3. Real-Time Focus and Practicality

Many studies focus on offline or academic IDS development, often failing to address the real-time requirements of IoT networks. These systems are unsuitable for practical deployment due to high latency, resource usage, and inability to adapt to dynamic environments. However, our proposed IDS is specifically designed for real-time operation, ensuring practical usability in real-world IoT networks.

4.2.4. Superior Accuracy and Performance

Studies relying on outdated datasets like KDD, NSL-KDD, or UNSW-NB15 typically report lower accuracy and lack relevance to IoT-specific challenges. Even works using the BoT-IoT dataset often fail to reach high accuracy due to insufficient preprocessing, inappropriate feature selection, and a lower volume of training datasets used. Achieving 99.2% accuracy with Random Forest surpasses most existing studies, even those using the BoT-IoT dataset. By addressing preprocessing and feature selection thoroughly, our model outperforms studies that rely on deep-learning models requiring greater computational resources and data volume.

4.2.5. Bridging the Gap Between Research and Deployment

Existing works often produce models that are either too complex (deep-learning-based, requiring significant computational power) or too simplistic (focusing only on basic attacks or outdated datasets). Our approach strikes the perfect balance using state-of-the-art machine learning that is both accurate and computationally efficient, making it feasible for deployment in resource-constrained IoT environments. The insights provided (e.g., Random Forest's suitability) are directly usable for researchers and practitioners looking to implement IDS solutions.

4.2.6. Significant Guidance for Future Research

Studies often focus on presenting their results without providing insights or guidelines for future researchers. Our detailed comparison and analysis of algorithms offer a clear roadmap for future researchers, helping them choose the best machine-learning models and strategies for IDS development in IoT networks. By emphasizing feature selection, attack diversity, and real-time detection, we establish a strong foundation for advancing future IDS research.

5. Conclusions and Deployment Architecture

In this paper, we have attempted to discover the most robust supervised classifier for building an accurate and high-performance network intrusion-detection system for real-time IoT networks. We have trained and evaluated the performance of seven state-of-the-art supervised machine-learning algorithms. We have explored a new and realistic BoT-IoT dataset in which different types of attack data have been injected. We extracted the most relevant features from the dataset using the Pearson Correlation Coefficient technique. To deeply examine their behavior, all models were trained three times using a different number of feature sets. The performance of algorithms was further analyzed and compared in terms of Precision, Recall, and F1 Score. The accuracy of Random Forest was the highest, at 99.2%, and the second-best performing algorithm was Naïve Bayes, giving an accuracy of 98.8%. Thus, from the experimental analysis, it is evident that the Random Forest algorithm is the most effective and most robust algorithm out of the seven machine-learning algorithms for intrusion detection in IoT networks. Our proposed INIDS can be deployed in IoT networks in the following ways:

Lightweight design: Our model's lightweight nature (after optimization) makes it feasible for integration into resource-constrained IoT devices.

Edge or fog layer: Our proposed IDS can be deployed at the edge or fog layer of IoT networks to monitor and analyze traffic locally. This minimizes latency and ensures the faster detection of threats in real time.

Cloud integration: The system can communicate with the cloud for centralized monitoring and management if necessary, enabling scalability for larger networks. However, we aim to integrate such a model with added performance into SDN-based networks.

Author Contributions: J.A. drafted the manuscript and did the experiments and G.M.R. proposed the main idea of the research and assisted in the experiments. B.-S.K. supervised the overall work and helped during the entire research and drafting of the paper. A.W. reviewed the paper and provided valuable guidance in paper writing and improving its quality. H.-Y.K. writing the original draft. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the Culture, Sports, and Tourism R&D Program through the Korea Creative Content Agency grant funded by the Ministry of Culture, Sports, and Tourism in 2024 (Project Name: Global Talent Training Program for Copyright Management Technology in Game Contents, Project Number: RS-2024-00396709, Contribution Rate: 100%).

Institutional Review Board Statement: Not Applicable.

Informed Consent Statement: Not Applicable.

Data Availability Statement: The data that support the findings of this study are openly available at the following URL <https://research.unsw.edu.au/projects/bot-iot-dataset> (accessed on 16 August 2024).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Li, P.; Zhang, Y. A novel intrusion detection method for internet of things. In Proceedings of the 2019 Chinese Control And Decision Conference (CCDC), Nanchang, China, 3–5 June 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 4761–4765.
2. Pathan, A.S.K. *The State of the Art in Intrusion Prevention and Detection*; CRC Press: Boca Raton, FL, USA, 2014.
3. de Souza, C.A.; Westphall, C.B.; Machado, R.B.; Sobral, J.B.M.; dos Santos Vieira, G. Hybrid approach to intrusion detection in fog-based IoT environments. *Comput. Netw.* **2020**, *180*, 107417. [\[CrossRef\]](#)
4. Liu, Z.; Thapa, N.; Shaver, A.; Roy, K.; Yuan, X.; Khorsandroo, S. Anomaly Detection on IoT Network Intrusion Using Machine Learning. In Proceedings of the 2020 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD), Durban, South Africa, 6–7 August 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–5.
5. Khraisat, A.; Gondal, I.; Vamplew, P.; Kamruzzaman, J. Survey of intrusion detection systems: Techniques, datasets and challenges. *Cybersecurity* **2019**, *2*, 20.
6. Koroniotis, N.; Moustafa, N.; Sitnikova, E.; Turnbull, B. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Gener. Comput. Syst.* **2019**, *100*, 779–796. [\[CrossRef\]](#)
7. Ikotun, A.M.; Ezugwu, A.E.; Abualigah, L.; Abuhaija, B.; Heming, J. K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Inf. Sci.* **2023**, *622*, 178–210. [\[CrossRef\]](#)
8. Shafiq, M.; Tian, Z.; Sun, Y.; Du, X.; Guizani, M. Selection of effective machine learning algorithm and Bot-IoT attacks traffic identification for internet of things in smart city. *Future Gener. Comput. Syst.* **2020**, *107*, 433–442. [\[CrossRef\]](#)
9. Shafiq, M.; Tian, Z.; Bashir, A.K.; Du, X.; Guizani, M. Corrauc: A malicious bot-iot traffic detection method in iot network using machine learning techniques. *IEEE Internet Things J.* **2020**, *8*, 3242–3254. [\[CrossRef\]](#)
10. Alkadi, O.; Moustafa, N.; Turnbull, B.; Choo, K.K.R. A deep blockchain framework-enabled collaborative intrusion detection for protecting iot and cloud networks. *IEEE Internet Things J.* **2020**, *8*, 9463–9472. [\[CrossRef\]](#)
11. Churcher, A.; Ullah, R.; Ahmad, J.; Masood, F.; Gogate, M.; Alqahtani, F.; Nour, B.; Buchanan, W.J. An Experimental Analysis of Attack Classification Using Machine Learning in IoT Networks. *Sensors* **2021**, *21*, 446. [\[CrossRef\]](#)
12. Kotsiantis, S.B.; Kanellopoulos, D.; Pintelas, P.E. Data preprocessing for supervised learning. *Int. J. Comput. Sci.* **2006**, *1*, 111–117.
13. Tufail, S.; Riggs, H.; Tariq, M.; Sarwat, A.I. Advancements and challenges in machine learning: A comprehensive review of models, libraries, applications, and algorithms. *Electronics* **2023**, *12*, 1789. [\[CrossRef\]](#)

14. Dini, P.; Elhanashi, A.; Begni, A.; Saponara, S.; Zheng, Q.; Gasmi, K. Overview on intrusion detection systems design exploiting machine learning for networking cybersecurity. *Appl. Sci.* **2023**, *13*, 7507. [\[CrossRef\]](#)
15. Bhavsar, M.; Roy, K.; Kelly, J.; Olusola, O. Anomaly-based intrusion detection system for IoT application. *Discov. Internet Things* **2023**, *3*, 5. [\[CrossRef\]](#)
16. Benesty, J.; Chen, J.; Huang, Y.; Cohen, I. Pearson correlation coefficient. In *Noise Reduction in Speech Processing*; Springer: Berlin/Heidelberg, Germany, 2009; pp. 1–4.
17. Goutte, C.; Gaussier, E. A probabilistic interpretation of precision, recall and F-score, with implication for evaluation. In *European Conference on Information Retrieval*; Springer: Berlin/Heidelberg, Germany, 2005; pp. 345–359.
18. Logeswari, G.; Bose, S.; Anitha, T. An intrusion detection system for sdn using machine learning. *Intell. Autom. Soft Comput.* **2023**, *35*, 867–880. [\[CrossRef\]](#)
19. Musleh, D.; Alotaibi, M.; Alhaidari, F.; Rahman, A.; Mohammad, R.M. Intrusion detection system using feature extraction with machine learning algorithms in IoT. *J. Sens. Actuator Netw.* **2023**, *12*, 29. [\[CrossRef\]](#)
20. Ahmed, M.R.; Shatabda, S.; Islam, A.M.; Robin, M.T.I. Intrusion Detection System in Software-Defined Networks Using Machine Learning and Deep Learning Techniques—A Comprehensive Survey. *Authorea Prepr.* **2023**, preprint.
21. Halbouni, A.; Gunawan, T.S.; Habaebi, M.H.; Halbouni, M.; Kartiwi, M.; Ahmad, R. Machine learning and deep learning approaches for cybersecurity: A review. *IEEE Access* **2022**, *10*, 19572–19585. [\[CrossRef\]](#)
22. Al-Shareeda, M.A.; Manickam, S.; Saare, M.A. DDoS attacks detection using machine learning and deep learning techniques: Analysis and comparison. *Bull. Electr. Eng. Inform.* **2023**, *12*, 930–939. [\[CrossRef\]](#)
23. Vanin, P.; Newe, T.; Dhirani, L.L.; O’Connell, E.; O’Shea, D.; Lee, B.; Rao, M. A study of network intrusion detection systems using artificial intelligence/machine learning. *Appl. Sci.* **2022**, *12*, 11752. [\[CrossRef\]](#)
24. Attou, H.; Guezzaz, A.; Benkirane, S.; Azrour, M.; Farhaoui, Y. Cloud-based intrusion detection approach using machine learning techniques. *Big Data Min. Anal.* **2023**, *6*, 311–320. [\[CrossRef\]](#)
25. Caruana, R.; Niculescu-Mizil, A. An empirical comparison of supervised learning algorithms. In Proceedings of the 23rd International Conference on Machine Learning, New York, NY, USA, 25–29 June 2006; pp. 161–168.
26. Alrashdi, I.; Alqazzaz, A.; Aloufi, E.; Alharthi, R.; Zohdy, M.; Ming, H. Ad-iot: Anomaly detection of iot cyberattacks in smart city using machine learning. In Proceedings of the 2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 7–9 January 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 305–310.
27. Shafiq, M.; Tian, Z.; Bashir, A.K.; Du, X.; Guizani, M. IoT malicious traffic identification using wrapper-based feature selection mechanisms. *Comput. Secur.* **2020**, *94*, 101863. [\[CrossRef\]](#)
28. Ge, M.; Fu, X.; Syed, N.; Baig, Z.; Teo, G.; Robles-Kelly, A. Deep learning-based intrusion detection for iot networks. In Proceedings of the 2019 IEEE 24th Pacific Rim International Symposium on Dependable Computing (PRDC), Kyoto, Japan, 1–3 December 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 256–25609.
29. Lo, W.W.; Layeghy, S.; Sarhan, M.; Gallagher, M.; Portmann, M. E-graphsage: A graph neural network based intrusion detection system for iot. In Proceedings of the NOMS 2022–2022 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, 25–29 April 2022; IEEE: Piscataway, NJ, USA, 2022; pp. 1–9.
30. Khan, R.U.; Zhang, X.; Alazab, M.; Kumar, R. An improved convolutional neural network model for intrusion detection in networks. In Proceedings of the 2019 Cybersecurity and cyberforensics conference (CCC), Melbourne, Australia, 8–9 May 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 74–77.
31. Sarwar, A.; Hasan, S.; Khan, W.U.; Ahmed, S.; Marwat, S.N.K. Design of an advance intrusion detection system for IoT networks. In Proceedings of the 2022 2nd international conference on artificial intelligence (ICAI), Islamabad, Pakistan, 30–31 March 2022; IEEE: Piscataway, NJ, USA, 2022; pp. 46–51.
32. Sharma, B.; Sharma, L.; Lal, C.; Roy, S. Anomaly based network intrusion detection for IoT attacks using deep learning technique. *Comput. Electr. Eng.* **2023**, *107*, 108626. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.